

0.1 Thu thập và xử lý dữ liệu

0.1.1 Dữ liệu phát hiện biển số

Tập dữ liệu dùng cho bài toán phát hiện biển số được tổ chức theo định dạng YOLOv8 OBB. Khác với bounding box ngang thông thường, mỗi nhãn trong tập dữ liệu gồm `class_id` lớp và bốn điểm của biển số:

```
<class_id> x1 y1 x2 y2 x3 y3 x4 y4
```

Các tọa độ được chuẩn hóa về khoảng $[0, 1]$. Cách biểu diễn này phù hợp với bài toán phát hiện biển số trong video, vì biển số thường bị nghiêng do góc quay camera, hướng di chuyển của phương tiện hoặc phối cảnh đường phố. Việc dùng OBB giúp vùng biển số được bao sát hơn, giảm phần nền thừa khi crop và tạo đầu vào ổn định hơn cho bước OCR phía sau.

Tập dữ liệu được tổng hợp từ nhiều nguồn khác nhau nhằm tăng độ đa dạng về góc nhìn, điều kiện ánh sáng, loại phương tiện và bố cục biển số. Các nguồn chính gồm dữ liệu Kaggle, các frame trích xuất từ video/YouTube, ảnh phương tiện từ Platesmania [1], và tập UFPR-ALPR. Ngoài ra, tập dữ liệu được bổ sung các mẫu âm tính không chứa biển số để giảm hiện tượng detector phát hiện nhầm biển số ở vùng không chứa biển.

Bảng 1: Thống kê dữ liệu phát hiện biển số

Nhóm dữ liệu	Số ảnh	Ghi chú
Platesmania [1]	26.836	Nguồn lớn nhất, ảnh phương tiện thực tế
Kaggle	4.578	Ảnh biển số xe chụp tại các bãi đỗ xe
Các nguồn video	2.851	Frame trích xuất từ video
UFPR-ALPR	4.500	Dùng để tăng đa dạng hình học cho bài toán định vị
Mẫu âm tính	1.700	Ảnh không chứa biển số, dùng để giảm false positive

Tổng cộng, tập dữ liệu gồm 40.465 ảnh và 48.431 nhãn OBB. Dữ liệu được chia thành 33.333 ảnh huấn luyện và 7.132 ảnh kiểm định, tương ứng khoảng 82,4% / 17,6%. Trong toàn bộ tập dữ liệu có 38.261 ảnh có ít nhất một biển số và 2.204 ảnh âm tính với file nhãn rỗng. Các ảnh âm tính này vẫn được giữ lại vì trong hệ thống thực tế, detector thường nhận đầu vào là crop phương tiện hoặc frame video có thể không chứa biển số rõ ràng; nếu chỉ huấn luyện trên ảnh dương tính, mô hình dễ sinh dự đoán sai trên các vùng như đèn xe, logo, mặt ca-lăng hoặc biển quảng cáo.

0.1.2 Dữ liệu nhận diện biển số

Tập dữ liệu dùng cho bài toán OCR biển số gồm hai phần train và valid. Mỗi ảnh là một crop biển số đã được tách khỏi ảnh phương tiện hoặc frame gốc. Nhãn OCR được mã hóa trực tiếp trong tên file: phần trước dấu # là chuỗi biển số, phần

sau dấu # là định danh mẫu. Các biển số phần lớn được crop từ ảnh phương tiện thu thập trên Platesmania [1]; phần còn lại đến từ Kaggle.

Ví dụ về tên nhãn dữ liệu:

79A[SEP]122.35#nomer12660750.jpg

30A-827.34#6748.jpg

KP-51-53#6542.jpg

Trong đó, [SEP] biểu diễn điểm ngắt dòng đối với biển số hai dòng. Cách mã hóa này phù hợp với mô hình OCR end-to-end, vì mô hình học trực tiếp ánh xạ từ ảnh crop sang chuỗi ký tự, không cần annotation bounding box cho từng ký tự.

Thống kê tổng quan của tập dữ liệu như sau:

Bảng 2: Thống kê tập dữ liệu OCR

Thuộc tính	Số lượng
Tổng số lượng	33.376
Train	27.841
Valid	5.535
Ảnh từ Platesmania [1]	18.925
Ảnh từ Kaggle	14.451
Số chuỗi biển số duy nhất	25.438

Tập dữ liệu có cả biển một dòng và biển hai dòng. Trong đó, biển hai dòng chiếm tỷ lệ lớn hơn:

Bảng 3: Thống kê bố cục

Kiểu bố cục	Số ảnh	Tỉ lệ
Một dòng	10.731	32,15%
Hai dòng	22.645	67,85%

Tập ký tự được sử dụng để huấn luyện mô hình gồm 10 chữ số (0–9), 23 chữ cái Latin viết hoa xuất hiện trên biển số xe Việt Nam (A, B, C, D, E, F, G, H, K, L, M, N, P, Q, R, S, T, U, V, X, Y, Z, Đ), cùng các ký tự phân tách (-, ., [SEP]). Ngoài ra, tập ký tự còn bao gồm ký hiệu <blank> phục vụ thuật toán CTC.

0.1.3 Dữ liệu đánh giá chất lượng biển số

Bên cạnh dữ liệu phát hiện và nhận diện ký tự, hệ thống còn sử dụng một tập dữ liệu riêng cho bài toán đánh giá chất lượng ảnh crop biển số. Mục tiêu của mô hình này là ước lượng mức độ đọc được của biển số trước khi đưa ảnh sang OCR hoặc các bước xử lý tăng cường. Việc tách riêng bài toán chất lượng giúp pipeline tránh xử lý mọi crop theo cùng một cách: ảnh rõ có thể được nhận diện trực tiếp,

trong khi ảnh mờ, nhỏ hoặc khó đọc có thể được đưa vào nhánh chờ thêm khung hình hoặc nhánh hợp nhất đa khung.

Ở giai đoạn đầu, mô hình được huấn luyện trên dữ liệu LPLCv2 do nhóm tác giả cung cấp. LPLCv2 là tập dữ liệu phân loại độ đọc được của biển số với bốn mức nhãn *perfect*, *good*, *poor* và *illegible*, được xây dựng nhằm đánh giá không chỉ chất lượng ảnh nói chung mà trực tiếp là khả năng đọc ký tự trên biển số [2]. Từ dữ liệu gốc, đồ án tạo bản dùng huấn luyện theo bốn lớp chất lượng và chia thành hai tập *train/valid* để huấn luyện mô hình ban đầu.

Bảng 4: Thống kê dữ liệu đánh giá chất lượng biển số

Tập dữ liệu	Train	Valid	Tổng số ảnh
LPLCv2-quality	39.767	14.876	54.643
Fine-tune nội bộ	24.796	5.993	30.789

Sau khi có mô hình khởi tạo từ LPLCv2, đồ án *fine-tune* thêm trên tập dữ liệu nội bộ được gom từ các crop biển số gần với miền dữ liệu của hệ thống hơn. Các ảnh này vẫn được tổ chức theo cùng bốn mức chất lượng, nhờ đó mô hình giữ được định nghĩa nhãn nhất quán nhưng thích nghi tốt hơn với ảnh biển số Việt Nam, điều kiện camera thực tế và các lỗi thường gặp trong pipeline như crop lệch, nhòe chuyển động hoặc biển số quá nhỏ.

Bảng 5: Phân bố nhãn trong dữ liệu chất lượng

Tập dữ liệu	Perfect	Good	Poor	Illegible
LPLCv2-quality	24.271	13.458	9.898	7.016
Fine-tune nội bộ	7.117	12.198	10.130	1.344

0.2 Huấn luyện và *fine-tuning* mô hình

0.2.1 Môi trường huấn luyện

Các mô hình được huấn luyện trên máy trạm trang bị GPU NVIDIA GeForce RTX 4050. Phần mềm huấn luyện dựa trên nền tảng Python 3.10, PyTorch 2.6.0 và CUDA 12.4.

0.2.2 Kế thừa mô hình phát hiện phương tiện

Trong hệ thống đề xuất, bước nhận diện phương tiện đóng vai trò tiền xử lý quan trọng nhằm xác định và định vị các đối tượng phương tiện giao thông trong khung hình trước khi tiến hành trích xuất biển số. Để thực hiện bước này, đồ án kế thừa và sử dụng bộ trọng số mô hình nhận diện phương tiện được công bố bởi Che et al. trong công trình "Character Time-series Matching for Robust License Plate Recognition" [3].

Trong phạm vi đồ án này, bộ trọng số pretrain Che et al. được sử dụng trực tiếp

mà không thực hiện fine-tuning thêm, với mục đích duy nhất là phát hiện phương tiện. Hệ thống cấu hình mô hình để nhận diện sáu lớp phương tiện phù hợp với bối cảnh giao thông Việt Nam, bao gồm: xe máy (class ID 1), xe con (class ID 6), xe tải (class ID 7), xe tải van (class ID 8), xe buýt (class ID 9), và xe ba gác chở hàng (class ID 10). Các lớp này được lọc trực tiếp từ đầu ra của mô hình thông qua danh sách class ID tương ứng ở bước hậu xử lý.

Việc tái sử dụng bộ trọng số này mang lại hai lợi thế chính. Thứ nhất, mô hình đã được huấn luyện trực tiếp trên dữ liệu phương tiện Việt Nam thu thập trong điều kiện thực tế, do đó có khả năng thích nghi tốt với đặc thù mật độ giao thông và điều kiện ánh sáng tại Việt Nam. Thứ hai, kiến trúc YOLOv5m với khoảng 21,2 triệu tham số cung cấp sự cân bằng hợp lý giữa độ chính xác và tốc độ suy luận, phù hợp với yêu cầu xử lý video thời gian thực của hệ thống.

0.2.3 Huấn luyện mô hình phát hiện biển số YOLOv8-OB

Mô hình phát hiện biển số YOLOv8-OB được huấn luyện theo tác vụ phát hiện hộp bao có hướng, tương ứng với cấu hình `task=obj` trong Ultralytics. Mô hình được khởi tạo với trọng số ngẫu nhiên theo cơ chế mặc định của Ultralytics/PyTorch, các thiết lập chính của quá trình huấn luyện được trình bày trong Bảng 6.

0.2.4 Huấn luyện và fine-tune Quality Router

Mô hình Quality Router được xây dựng dựa trên kiến trúc phân loại ảnh nhẹ YOLOv8n-cls. Quá trình huấn luyện sử dụng tập dữ liệu đánh giá chất lượng được trình bày ở Mục 0.1.3 và diễn ra theo hai giai đoạn: đầu tiên mô hình được huấn luyện trên dữ liệu LPLCv2 để học khái niệm độ đọc được (legibility) tổng quát, sau đó được fine-tune trên tập ảnh crop biển số thực tế tại Việt Nam nhằm thích nghi với miền dữ liệu của hệ thống. Mô hình duy trì cấu trúc dự đoán bốn nhãn phân loại: *perfect*, *good*, *poor* và *illegible*.

Bên cạnh bài toán bốn lớp, một biến thể nhị phân cũng được huấn luyện như một thí nghiệm phụ, tuy nhiên hệ thống chốt sử dụng mô hình bốn lớp để phục vụ cơ chế ánh xạ ba nhánh (`direct`, `tracklet_fusion`, `unreadable_wait`) trong pipeline. Cấu hình huấn luyện chi tiết cho giai đoạn fine-tune được trình bày trong Bảng 7.

Kích thước đầu vào 64×64 được lựa chọn để tối ưu tốc độ suy luận, phù hợp với yêu cầu xử lý video thời gian thực. Các kỹ thuật tăng cường dữ liệu đặc thù như Random Erasing (`erasing=0.4`) và RandAugment được áp dụng nhằm mô phỏng các biến dạng lóa sáng, nhiễu hạt và che khuất một phần thường gặp trên ảnh crop biển số, giúp mô hình tăng cường khả năng khái quát hóa. Chi tiết về độ chính xác (Accuracy) và ma trận nhầm lẫn của mô hình được đánh giá trong Mục ??.

Bảng 6: Cấu hình huấn luyện mô hình phát hiện biến số dạng OBB

Nhóm cấu hình	Giá trị
Tác vụ	task=obb
Checkpoint khởi tạo	Khởi tạo ngẫu nhiên
Kích thước ảnh đầu vào	640 × 640
Số epoch	50
Batch size	8
Thiết bị huấn luyện	GPU, device=0
Số workers	8
AMP	True
Thiết lập tái lập	seed=0, deterministic=True
Bộ tối ưu	auto
Learning rate	lr0=0.001, lrf=0.01
Momentum và weight decay	momentum=0.937, weight_decay=0.0005
Warm-up	warmup_epochs=3, warmup_momentum=0.8, warmup_bias_lr=0.1
Trọng số loss	box=7.5, cls=0.5, dfl=1.5, angle=1.0
Tăng cường dữ liệu	hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, translate=0.1, scale=0.5, flipplr=0.5, mosaic=1.0
Các phép không sử dụng	mixup, cutmix, copy_paste, flipud, shear, perspective
Close mosaic	close_mosaic=10

Bảng 7: Cấu hình huấn luyện mô hình Quality Router (YOLOv8n-cls)

Nhóm cấu hình	Giá trị
Tác vụ	<code>task=classify</code>
Kiến trúc backbone	<code>YOLOv8n-cls</code>
Kích thước ảnh đầu vào	<code>64 × 64</code>
Số epoch	<code>50 (Early stopping với patience=10)</code>
Batch size	<code>16</code>
Bộ tối ưu	<code>auto (lr0=0.0001, lrf=0.01)</code>
Momentum và weight decay	<code>momentum=0.937,</code> <code>weight_decay=0.0005</code>
Warm-up	<code>warmup_epochs=3.0,</code> <code>warmup_momentum=0.8,</code> <code>warmup_bias_lr=0.0</code>
Tăng cường dữ liệu	<code>hsv_h=0.015, hsv_s=0.7,</code> <code>hsv_v=0.4, translate=0.1,</code> <code>scale=0.5, flip_lr=0.5</code>
Auto-augment & Erasing	<code>auto_augment=randaugment, erasing=0.4</code>
Thiết bị & Môi trường	<code>GPU, device=0, 8 workers, AMP True</code>
Tính tái lập	<code>seed=0, deterministic=True</code>

0.2.5 Huấn luyện SmallLPR-Line-CTC

Mô hình SmallLPR-Line-CTC được huấn luyện bằng PyTorch Lightning trên dữ liệu OCR biển số Việt Nam. Quá trình huấn luyện sử dụng Trainer với accelerator tự động chọn CUDA, một device, precision 32, tối đa 200 epoch và gradient clipping. Seed toàn cục được cố định để đảm bảo tính tái lập. Cấu hình được quản lý tập trung qua file YAML.

Ba cơ chế theo dõi và kiểm soát được sử dụng: lưu top-3 checkpoint có độ chính xác kiểm định cao nhất và checkpoint cuối cùng vào thư mục có timestamp; dừng sớm nếu độ chính xác kiểm định không cải thiện trong 40 epoch liên tiếp; ghi lại learning rate mỗi epoch. Logging được thực hiện qua CSVLogger, ghi lại loss từng nhánh CTC, layout loss, layout accuracy và sequence accuracy.

Các tham số huấn luyện được trình bày trong Bảng 8.

0.3 Xây dựng Web App tích hợp pipeline nhận dạng

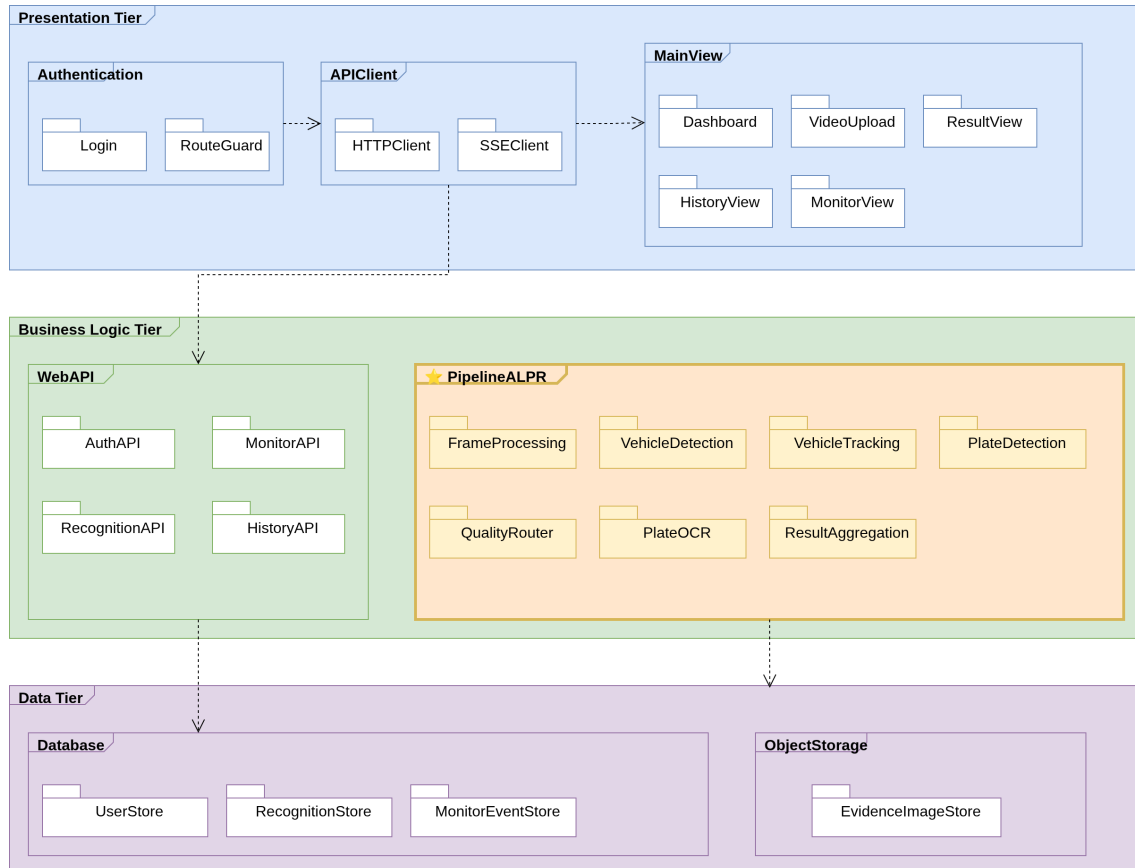
Hệ thống phần mềm được xây dựng nhằm cung cấp một kiến trúc backend linh hoạt và một giao diện trực quan, giúp người dùng cuối dễ dàng tương tác và sử dụng các tính năng của pipeline nhận dạng. Thông qua đó, hệ thống cung cấp các giao thức nạp dữ liệu video, giám sát tiến trình xử lý, đối soát kết quả nhận dạng và truy xuất cơ sở dữ liệu bằng chứng một cách thuận tiện.

Bảng 8: Cấu hình huấn luyện SmallLPR-Line-CTC

Nhóm	Tham số	Giá trị
DataLoader	Batch size Số workers Pin memory Shuffle (train) Shuffle (valid)	64 4 Có Có Không
Trainer	Accelerator Số devices Precision Số epoch tối đa Gradient clipping Optimizer Learning rate Scheduler Seed toàn cục File cấu hình	Tự động (CUDA nếu có) 1 32 200 1.0 AdamW 5×10^{-4} (khởi tạo backbone và scratch), 5×10^{-6} (decoder pretrained) CosineAnnealingWarmRestarts ($T_0 = 20$, $T_{mult} = 2$, $\eta_{min} = 10^{-6}$) 42 YAML
Callback	Lưu checkpoint Dừng sớm LR Monitor CSVLogger	Top-3 checkpoint có val_acc cao nhất patience = 40 epoch Ghi learning rate mỗi epoch Loss từng nhánh CTC, layout loss, layout accuracy, sequence accuracy

0.3.1 Thiết kế kiến trúc

Hệ thống được tổ chức theo kiến trúc ba tầng (Three-tier Architecture) kết hợp mô hình Client–Server. Ba tầng gồm tầng trình bày, tầng xử lý nghiệp vụ và tầng dữ liệu. Cách phân chia này giữ giao diện và cơ chế lưu trữ tách khỏi mã suy luận, trong khi toàn bộ mô hình học sâu và các thuật toán xử lý theo thời gian được đặt tại tầng nghiệp vụ. Kiến trúc gói của hệ thống được trình bày trong Hình 0.1.



Hình 0.1: Kiến trúc gói của hệ thống Web App theo mô hình ba tầng

Tầng trình bày (Presentation Tier). Tầng trình bày chịu trách nhiệm tiếp nhận thao tác và hiển thị trạng thái xử lý, không thực hiện suy luận mô hình. Package *Authentication* quản lý màn hình đăng nhập và bảo vệ định tuyến. Package *MainView* gồm các màn hình tải video, giám sát nguồn phát, xem kết quả và tra cứu lịch sử. Mọi trao đổi với backend được tập trung tại package *APIClient*: *HTTPClient* thực hiện các yêu cầu đồng bộ, còn *SSEClient* tiếp nhận tiến độ và kết quả nhận dạng theo luồng. Việc tạo một cổng giao tiếp duy nhất giúp giao diện không phụ thuộc vào cách các mô hình được tổ chức hoặc thay thế ở phía máy chủ.

Tầng xử lý nghiệp vụ (Business Logic Tier). Đây là tầng trung tâm của hệ thống và cũng là nơi pipeline nhận dạng được tích hợp. Package *WebAPI* cung cấp

bốn nhóm chức năng: *AuthAPI* xử lý xác thực; *RecognitionAPI* tiếp nhận video và tạo công việc nhận dạng; *MonitorAPI* quản lý nguồn giám sát cùng các trích đoạn được đánh dấu; *HistoryAPI* truy xuất các phiên đã xử lý. WebAPI chỉ đảm nhiệm kiểm tra yêu cầu, quản lý vòng đời công việc và chuyển dữ liệu; phần tính toán AI được ủy quyền cho package *PipelineALPR*.

Package *PipelineALPR* đóng gói các mô hình và thuật toán thành bảy module: *FrameProcessing*, *VehicleDetection*, *VehicleTracking*, *PlateDetection*, *QualityRouter*, *PlateOCR* và *ResultAggregation*. Luồng phối hợp tổng thể giữa các module này được trình bày tại Hình ?? ở Chương 3. Các mô hình phát hiện phương tiện, phát hiện biển số, phân loại chất lượng và nhận dạng ký tự đều là tài nguyên của tầng nghiệp vụ; chúng không được gọi trực tiếp từ giao diện. Cách đóng gói này biến chuỗi xử lý đã thiết kế ở Chương 3 thành một dịch vụ nhận dạng độc lập: WebAPI chỉ cần cung cấp nguồn khung hình và hàm nhận sự kiện, còn PipelineALPR tự quản lý detection, tracking, định tuyến chất lượng, OCR và tổng hợp đa khung hình.

Tầng dữ liệu (Data Tier). Tầng dữ liệu được chia thành package *Database* và *ObjectStorage*. Database lưu tài khoản, metadata phiên xử lý, kết quả nhận dạng và sự kiện giám sát thông qua các kho *UserStore*, *RecognitionStore* và *MonitorEventStore*. Ảnh phương tiện, biển số và ảnh bằng chứng được tách sang *EvidenceImageStore*; cơ sở dữ liệu chỉ giữ URL tham chiếu. Do đó, pipeline có thể phát kết quả ngay khi suy luận hoàn tất, còn thao tác lưu metadata và ảnh bằng chứng được thực hiện tại ranh giới với tầng dữ liệu thay vì xen vào các bước tính toán mô hình.

Cơ chế tích hợp và vòng đời xử lý. Khi dịch vụ khởi động, các trọng số phát hiện phương tiện, phát hiện biển số, đánh giá chất lượng và nhận dạng ký tự được nạp một lần vào bộ nhớ và dùng chung giữa các phiên xử lý. Thiết kế này loại bỏ chi phí khởi tạo mô hình cho từng yêu cầu và hạn chế việc nhân bản dữ liệu trên VRAM. Ngược lại, các thành phần có trạng thái như bộ theo vết, bộ liên kết biển số–phương tiện và bộ đệm OCR được khởi tạo riêng cho từng phiên. Nhờ đó, định danh quỹ đạo và bằng chứng của các video xử lý đồng thời không bị trộn lẫn.

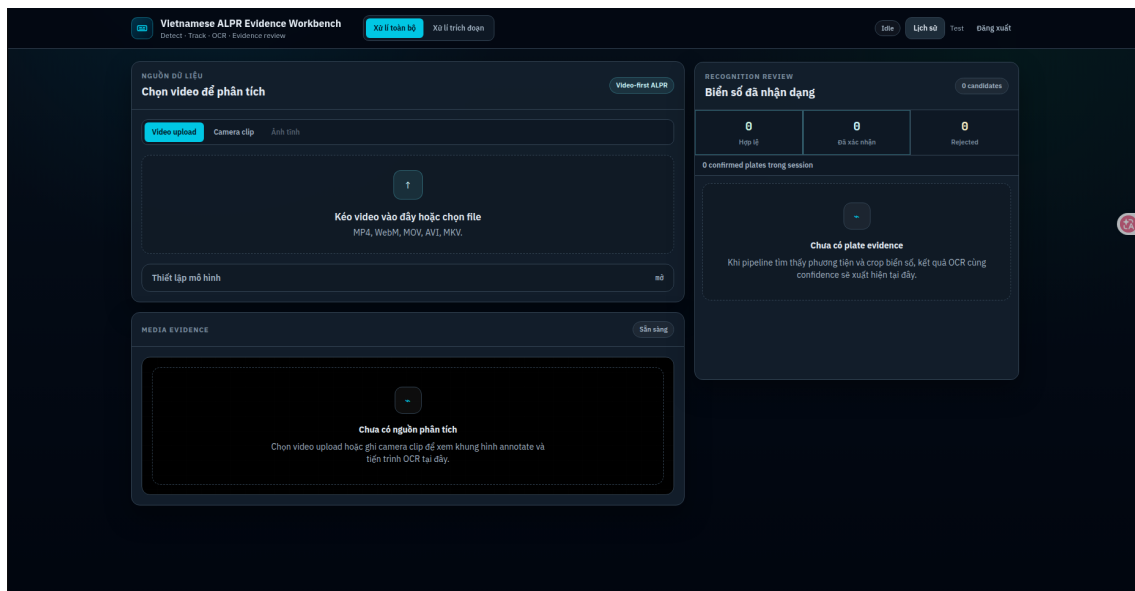
Hai đầu vào nghiệp vụ chính được hội tụ về cùng một lõi xử lý. Với chế độ xử lý toàn bộ, *RecognitionAPI* tạo nguồn khung hình tuần tự và khởi chạy pipeline trong một worker nền. Với chế độ xử lý trích đoạn, *MonitorAPI* duy trì bộ đệm vòng chứa các khung hình gần nhất; khi người dùng đánh dấu một trích đoạn, cửa sổ thời gian tương ứng được chuyển thành cùng kiểu nguồn khung hình. Từ điểm này, cả hai

luồng đều đi qua PipelineALPR và sử dụng chung cơ chế phát hiện, theo vết, OCR và tổng hợp kết quả.

Kết quả trung gian và kết quả cuối được chuyển ngược tới WebAPI dưới dạng sự kiện. WebAPI đưa các sự kiện này vào hàng đợi bất đồng bộ để truyền tiến độ, ảnh xem trước và kết quả nhận dạng tới *SSEClient*; đồng thời chuyển kết quả hoàn chỉnh sang tầng dữ liệu để lưu metadata và ảnh bằng chứng. Số công việc suy luận chạy đồng thời được giới hạn theo tài nguyên GPU, giúp tránh tràn VRAM khi nhiều yêu cầu đến cùng lúc. Như vậy, phần Web App không chỉ bao quanh pipeline bằng giao diện, mà còn cung cấp cơ chế đóng gói mô hình, cô lập trạng thái phiên, điều phối tác vụ GPU và lưu vết kết quả phục vụ kiểm chứng.

0.3.2 Thiết kế giao diện người dùng

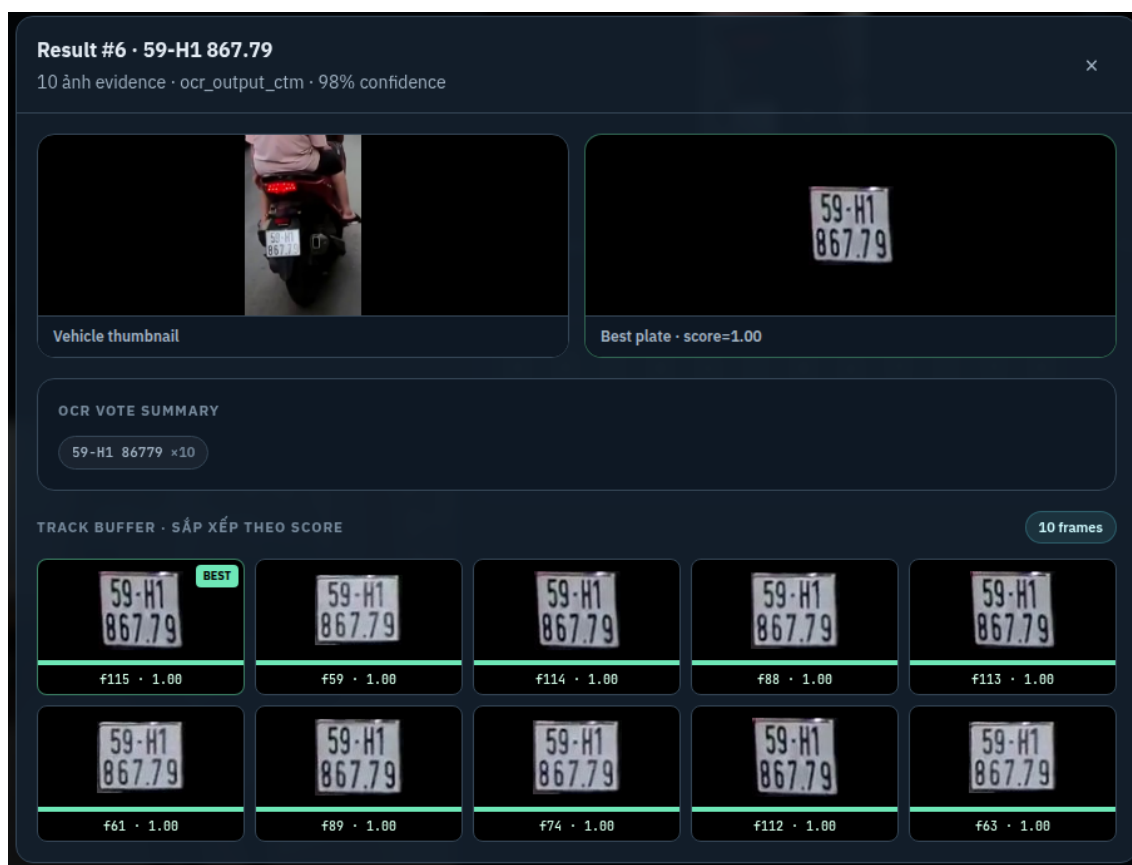
Giao diện người dùng (GUI) được thiết kế theo hướng phân chia module, gồm hai phân hệ chính: module quản lý nguồn cấp dữ liệu và module đối soát kết quả. Phân hệ nguồn cấp dữ liệu hỗ trợ tải lên video, giám sát luồng phát trực tiếp, định tuyến sự kiện, và cung cấp các chỉ số theo dõi tiến trình trực quan. Phân hệ kết quả đóng vai trò như một bảng điều khiển (dashboard), truy xuất và hiển thị dữ liệu chi tiết bao gồm: chuỗi biển số định dạng văn bản, nhãn phương tiện, điểm độ tin cậy, các vùng cắt (crop) đối tượng và tập hợp bằng chứng OCR theo chuỗi thời gian.



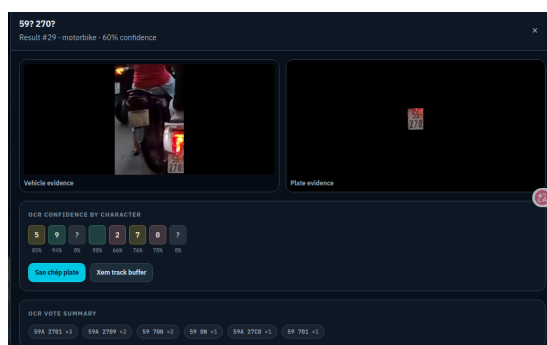
Hình 0.2: Giao diện tổng quan của hệ thống phần mềm

Các mẫu dữ liệu bị mô hình đánh giá chất lượng từ chối cũng được hệ thống ghi vết và lưu trữ độc lập, phục vụ mục đích gỡ lỗi (debugging) và phân tích nguyên nhân thay vì loại bỏ mà không lưu vết (silent drop).

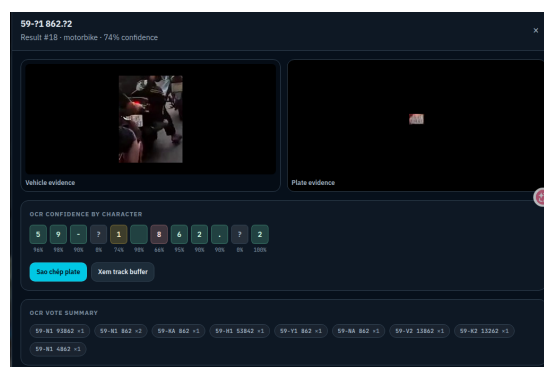
Quá trình giao tiếp giữa máy chủ và giao diện người dùng được duy trì thông



Hình 0.3: Bảng điều khiển chi tiết truy xuất kết quả nhận dạng phương tiện và chuỗi bằng chứng OCR



(a) Trường hợp 1: Biển bị cắt thiếu ký tự

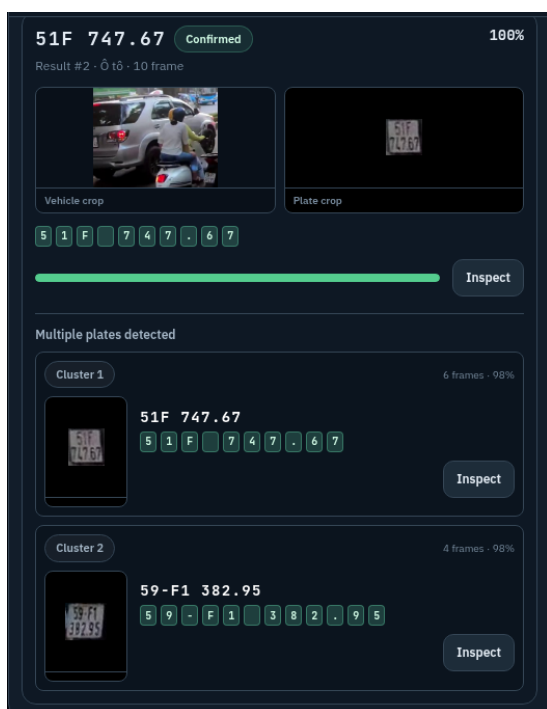


(b) Trường hợp 2: Biển số quá mờ, mất thường không nhìn rõ

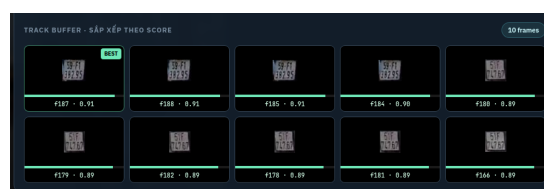
Hình 0.4: Giao diện ghi vết các trường hợp khung hình biển số bị từ chối

qua giao thức truyền phát sự kiện một chiều, cho phép cập nhật trạng thái hệ thống theo thời gian thực. Điều này giúp hệ thống phản hồi tức thời ngay khi một phương tiện hoàn tất chu kỳ nhận dạng. Tính năng truy xuất lịch sử phiên xử lý chủ yếu đóng vai trò công cụ kiểm chứng (validation) và trình diễn công nghệ.

Bên cạnh đó, giao diện cho phép kiểm tra trực tiếp bộ đệm chứng cứ để phân tích các tình huống ngoại lệ như hiện tượng ID Switch, giúp kỹ sư dễ dàng can thiệp và đánh giá hiệu quả của thuật toán phân cụm.



(a) Hiện tượng ID Switch



(b) Bộ đệm bằng chứng OCR phân cụm

Hình 0.5: Giao diện phân tích bộ đệm bằng chứng để gỡ lỗi hiện tượng ID Switch

0.3.3 Các kịch bản vận hành thực tiễn

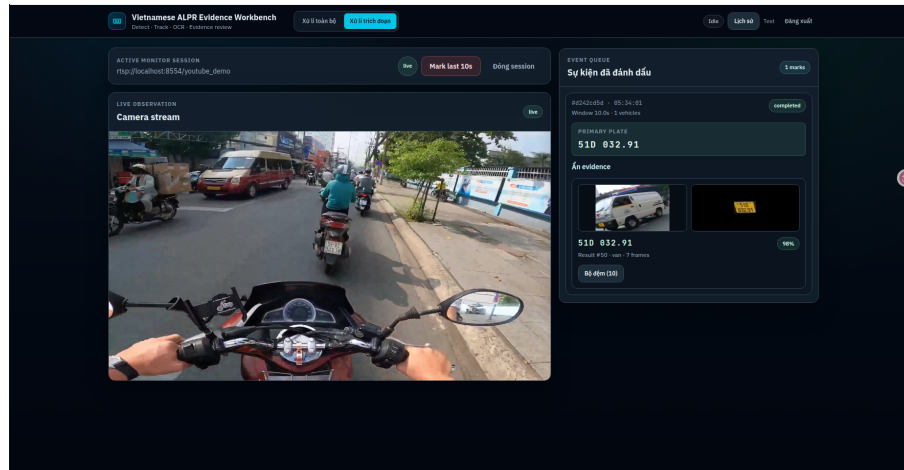
Hệ thống phần mềm hỗ trợ hai kịch bản vận hành chủ đạo, tương ứng với các tình huống triển khai trong thực tế:

1. **Phân tích toàn bộ video (Full-Video Processing).** Chức năng này cho phép người dùng tải lên và phân tích các tệp tin video ngắn (thời lượng tối đa 20 giây). Sau bước kiểm duyệt tính hợp lệ, hệ thống khởi tạo luồng xử lý và phân tích toàn diện trên tất cả các khung hình. Tiến trình nhận dạng phương tiện được đồng bộ hóa và báo cáo liên tục song song với luồng giải mã video.
2. **Phân tích trích đoạn (Clip-based Processing).** Cơ chế này được thiết kế nhằm tối ưu hóa tài nguyên tính toán, chỉ tập trung xử lý các khoảnh khắc có sự kiện thay vì phân tích dư thừa toàn bộ thời lượng:

- *Đối với luồng phát trực tiếp (Streaming):* Hệ thống liên tục giải mã và

cập nhật bộ đệm vòng cục bộ. Khi có thao tác đánh dấu từ người dùng, hệ thống mặc định trích xuất phân đoạn 10 giây tính lùi từ thời điểm đánh dấu để đưa vào pipeline phân tích.

- *Đối với video ngoại tuyến thời lượng dài:* Hệ thống cho phép người dùng tùy chọn đánh dấu các trích đoạn có độ dài tối đa 30 giây để tiến hành nhận dạng chuyên sâu.



Hình 0.6: Giao diện thao tác đánh dấu sự kiện và trích xuất phân đoạn luồng trực tiếp

Sản phẩm đầu ra của mỗi quỹ đạo phương tiện được đóng gói thành các đối tượng dữ liệu bao gồm định danh, chuỗi biển số, và tài liệu minh chứng dạng hình ảnh. Dữ liệu văn bản được lưu trữ có cấu trúc trong cơ sở dữ liệu MongoDB, trong khi dữ liệu hình ảnh được trừu tượng hóa qua kho lưu trữ đối tượng (Object Storage) và tham chiếu thông qua đường dẫn. Các lỗi ngoại lệ (exceptions) sinh ra từ một phiên xử lý được cô lập hoàn toàn (sandboxed) và phản hồi chính xác về giao diện gọi hàm tương ứng, qua đó giảm nguy cơ lỗi của một phiên xử lý lan sang các phiên khác.

0.4 Môi trường triển khai

Ứng dụng được triển khai theo mô hình tách riêng frontend, backend xử lý AI và các dịch vụ lưu trữ. Frontend được triển khai trên Vercel, backend được đóng gói bằng Docker và chạy trên RunPod GPU. Cơ sở dữ liệu và lưu trữ ảnh kết quả sử dụng các dịch vụ bên ngoài để giảm tải cho server xử lý chính.

Trong môi trường thử nghiệm cục bộ, hệ thống được chạy bằng Docker Compose gồm bốn dịch vụ chính: backend API, frontend web, MediaMTX và MongoDB. Backend mở cổng 8000, frontend mở cổng 3000, MongoDB mở cổng 27017, MediaMTX sử dụng các cổng 8554, 8889, 8189/udp và 9997. Cấu hình này phục vụ kiểm thử chức năng đăng nhập, tải video, xử lý nhận dạng và xem kết quả trực tiếp.

Thành phần	Cấu hình triển khai
Frontend	React/Vite, triển khai trên Vercel
Backend API	FastAPI, chạy trên cổng 8000
Server xử lý AI	RunPod GPU, NVIDIA RTX 3090, 24GB VRAM
Môi trường GPU	PyTorch 2.4.1, CUDA 12.1, cuDNN 9
Streaming video	MediaMTX
Cơ sở dữ liệu	MongoDB Atlas
Lưu trữ ảnh kết quả	Supabase Storage

Bảng 9: Cấu hình triển khai ứng dụng

Trong môi trường triển khai thử nghiệm trên server GPU, backend và MediaMTX được đóng gói trong cùng một Docker image. Backend đảm nhiệm các bước phát hiện phương tiện, phát hiện biển số, tracking, OCR và trả kết quả qua API. MediaMTX phục vụ luồng video trực tiếp, hỗ trợ hiển thị nguồn camera trong chế độ chọn trích đoạn trên giao diện web. Các mô hình như phát hiện phương tiện, phát hiện biển số, OCR và quality router được đặt trên network volume của RunPod.